Softwarica COLLEGE OF IT & E-COMMERCE | in collaboration with Coventry University

Student Assignment Brief

CONFIDENTIAL DOCUMENT

This document is intended solely for Softwarica College of IT & E-Commerce students for their own use in completing their assessed work for this module. It must not be passed to third parties or posted on any website.

Contents

Assignment Information

Module Name:	The Internet and Web Technologies
Module Code:	ST5041CMD
Assignment Title:	ST5041CMD The Internet and Web Technologies March Intake 2026 Coursework CW (Regular)
Assignment Due:	Jul 27, 2026

Assignment Credit:	20
Word Count:	1500
Assignment Type:	Coursework : Individual Report
Grading:	Percentage Grade(Applied Core Assessment).

Assessment Overview

You will be provided with an overall grade between 0% and 100%. You have one opportunity to pass the assignment at or above 40%.

Important Notice

The work you submit for this assignment must be your **own independent work**. More information is available in the 'Assignment Task' section of this assignment brief.

Assessed Module Learning Outcomes

The Learning Outcomes for this module align with the marking criteria which can be found at the end of this brief. Ensure you understand the marking criteria to ensure successful achievement of the assessment task. The following module learning outcomes are assessed in this task

On completion of this module the student should be able to:

1. Describe and discuss the core principles of user and session management work in systems accessed remotely.
2. Make use of the main web development technologies.
3. Develop a web application integrating f rontend and backend technologies.
4. Demonstrate awareness of web protocols

Assignment Task

For this coursework, students are required to specify, design, and develop a simple web application with user and session management. The scope of this individual project must be approved by the module leader prior to its development. Students are expected to demonstrate professional use of template-based frontend technologies (such as Jinja2) combined with HTML, CSS, and JavaScript. For the design and development of the server-side application, students are required to use Flask

API (Python). For data persistence, students must use an appropriate database system (MySQL) as part of the backend.

Project Selection

For this assignment, you must design and implement your own simple web application. You are free to choose your own idea for this project.

Example project ideas include (but are not limited to):

1. TaskManager – A basic to-do list application with user login
2. NoteKeeper – A personal note-saving application with authentication
3. EventPlanner – A simple event scheduling application
4. ContactSaver – Store and manage personal contacts
5. DailyJournal – A private journal application with secure login

Project Requirements

1. The project must cover the following points:
2. You must be creative in designing your web application that implements user and session management.
3. You must consider security architecture and patterns, along with secure design principles, while designing your web application.
4. You must use tools and technologies taught during this module such as: **HTML,CSS, JavaScript, Template engines (e.g. Jinja2)**
5. You must use Flask API for the backend server-side application.
6. You must use a database system for data persistence in the backend.
7. All your source code must be version controlled from the beginning using a version control system (Git).
8. The source code repository must be hosted in your personal GitHub repository.
9. The GitHub repository link must be submitted via the https://www.c4mpus.com assignment submission section before the specified deadline.

Video Demonstration Requirement

You must create a project demonstration screencast video with the following requirements:

1. Duration: Approximately 8 minutes
2. The video must include clear audio explanation covering:1. Core features of the application2. User and session management3. Security mechanisms4. System architecture and design decisions
3. The video must be uploaded to YouTube as “Unlisted” or uploaded to Google Drive.
4. The YouTube video link must be included in the README document of your GitHub repository

Report Requirement

You must submit a report of up to 1500 words documenting your work.

The report should include the following sections:

1. Project Overview
2. Technologies Used
3. Design and Architecture
4. Database Design and Backend Logic
5. Security Measures
6. Version Control Summary
7. Challenges Faced and Improvements

Important:
The links to your GitHub repository and project demonstration video must be included in the Version Control section of the report.

Notes Academic Regulations

1. A mandatory VIVA session will be conducted at the end of the semester to evaluate understanding of the project. Postponement is only permitted under exceptional circumstances with approval from the Course Director.
2. APA 7th edition referencing style must be used. Support is available via the Centre for Academic Writing (CAW).
3. Students requiring disability or learning support should notify the registry course support team and the module leader in advance.
4. The College is not responsible for loss or corruption of coursework stored on personal devices. Regular backups and use of github are strongly advised.
5. If technical issues prevent submission on the deadline day, a short extension (normally 24 hours or the next working day) may be granted and communicated by the module leader.
6. Collusion, plagiarism, and self-plagiarism are treated as serious academic misconduct and will be reported to the Academic Conduct Panel.
7. If there is a significant mismatch between coursework quality and in-class performance, students may be required to attend an in-person verification examination.
8. Submission of previously assessed work (either in full or in part) for another module or qualification is strictly prohibited and will be treated as self-plagiarism.

For resit or reassessment cases only, students may be permitted to reuse elements of a previous submission only with explicit approval from the module leader. In such cases:

1. The resubmitted work must demonstrate significant modification, improvement, and updating based on feedback provided.
2. Simply resubmitting the original work, or making superficial changes, is not acceptable.
3. Any reused content, ideas, or code from the previous attempt must be clearly identified and properly cited in accordance with APA 7th edition referencing guidelines.
4.

Students may be required to explain the changes made during the VIVA or verification process to demonstrate ownership and understanding of the updated work.

Submission Instructions

Requirement	Details
File Naming	NAME_studentID
File Format	.docx/.pdf format
Submission Method	Campus 4.0 platform (submission link provided 2 weeks before deadline)

Marking and Feedback

How will my assignment be marked?

Your assignment will be marked by the Module Team using standardized criteria.

How will I receive grades and feedback?

Provisional marks will be released once internally moderated. Feedback will be provided alongside grades release within 2 weeks (10 working days).

What will I be marked against?

Details of the marking criteria for this task can be found in the Assessment Marking Criteria section at the end of this brief.

Grade Requirements

You must achieve 40% or above to pass this assessment. Ensure you understand the marking criteria for successful completion.

Assignment Support and Academic Integrity

Getting Help

If you have any questions about this assignment, please meet your respective module leader or teacher for more information.

Language Standards

You are expected to use effective, accurate, and appropriate language within this assessment task.

Academic Integrity

The work you submit must be your own. All sources of information need to be acknowledged and attributed; therefore, you must provide references for all sources of information and acknowledge any tools used in the production of your work, **excluding Artificial Intelligence (AI)** .
We use detection software and make routine checks for evidence of academic misconduct. Definitions of academic misconduct, including plagiarism, self-plagiarism, and collusion can be found in Student handbook in Campus 4.0.
All cases of suspected academic misconduct are referred to for investigation, the outcomes of which can have profound consequences to your studies.

Support for Students with Disabilities

If you have a disability, long-term health condition, specific learning difference, mental health diagnosis or symptoms, contact the Student Support Office for assistance.

Unable to Submit on Time?

If events prevent you from submitting on time, guidance on extenuating circumstances is available in the Student Handbook or from the Student Support Office.

Administration of Assessment

Module Leader Name:	_
Module Leader Email:	_
Assignment Category:	
Attempt Type:	regular
Component Code:	cw

Assessment Marking Criteria

Criteria	0	1-3	4-6	7-9	10-12	13-15
Design and-implementation (15 Marks)	Not Submitted	Poor visual design with cluttered layout, inconsistent fonts or colors, and confusing navigation.	Basic layout and styling present, but design is inconsistent and usability is limited.	Clear layout with readable content, logical navigation, and acceptable user experience.	Well-designed interface with consistent styling, reusable components, and good responsiveness.	Professional-quality UI with responsive design, strong UX principles, and polished visuals across devices.
	0	1-3	4-6	7-9	10-12	13-15
Persistence (15 Marks)	Not Submitted	No database or incorrect use of data storage.	Basic data storage with limited validation and poor schema design.	Functional database schema with basic relationships and constraints.	Well-organized, normalized database with appropriate data validation.	Optimized database design with efficient queries, indexing, and scalability considerations.
	0	1-3	4-6	7-9	10-12	13-15
Server-side programming (15 Marks)	Not Submitted.	Very limited backend functionality with minimal routing.	Basic server-side logic with simple routes and responses.	Functional backend with CRUD operations and database integration.	Well-structured Flask API application with authentication and proper routing.	Advanced backend implementation with architecture, modular design, and robust error handling.
	0	1-3	4-6	7-9	10-12	13-15
Security (15 Marks)	Not Submitted	No security measures implemented.	Minimal security with weak validation and insecure practices.	Basic security measures such as input validation and access control.	Proper use of password hashing, authentication, and secure configurations.	Advanced security including role-based access control, secure environment variables, and token/session management.
	0	1-2	3-4	5-6	7-8	8-10
Video and Version Control (10 Marks)	No video or version control used.	Poor-quality video and minimal or disorganized Git usage.	Partial demonstration of features with limited commit history.	Clear explanation of main features and reasonable commit usage.	Well-structured video and meaningful commit messages showing progress.	Professional-quality video and excellent version control practices including clear commits, branches, and development stages.
	0	1-4	5-8	9-12	13-16	17-20
	Not Submitted.					

Documentation(20 Marks)		Very poor documentation with missing or unclear sections.	Basic documentation covering some system features and setup steps.	Clear documentation explaining installation, usage, and main features.	Well-structured documentation with diagrams, architecture explanations, and technical details.	Professional-level documentation including purpose, setup, API descriptions, testing notes, screenshots, and limitations.
Testing (10 marks)	0	1-2	3-4	5-6	7-8	9-10
	No testing evidence provided.	Very limited informal testing with no documentation.	Basic manual testing of core functionality.	Documented test cases covering main features.	Well-organized testing including unit or API tests with clear results.	Comprehensive testing strategy with automated tests, edge cases, and clear reporting of results.